Laboratory 2

Exploring DSP concepts with STM32CubeIDE and the STM32F411E DISCO and MAC/SIMD

1. PREREQUISITES

- You have the setup as required in Lab 1 and the **STM32Cube Programmer** program installed.
- You have set up the supporting Jupyter Notebook in Google Colab or locally
- You have your own set of headphones (in case you want to listen to things in the Python notebook)
- Recommended programs: Audacity
- Check D2L for other materials that might be required

2. OUTLINE

In this lab, the aim is to explore some of the concepts around optimizing the C code based on understanding what is happening on the processor and then trying to use the features available on the processor. Unlike the previous lab, this activity will have a bit less handholding.

By the end of this lab, you should be:

- Comfortable with using the ITM port and SWV Trace Log to profile your code
- Able to use the STM32CubeProgrammer to load data in and out of memory
- Be able to optimize filter code using a variety of techniques, including the MAC and SIMD

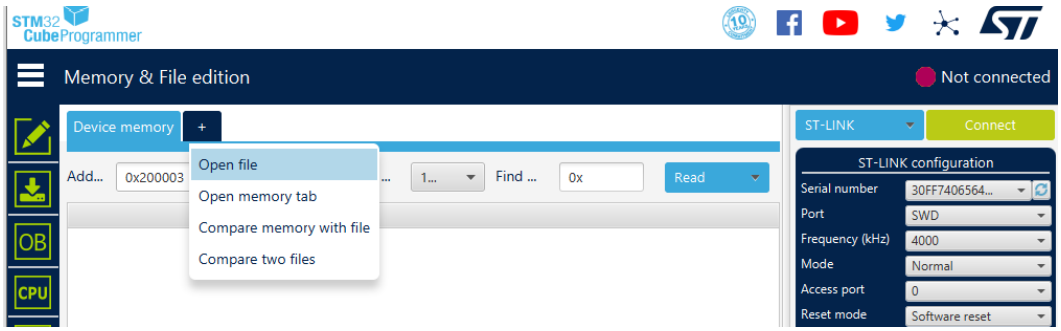
Q: There are several Questions throughout the lab for you to answer. You should collate your answers/observations/screenshots in a Word document (or equivalent), such as the lab sheet available on D2L

PART ONE

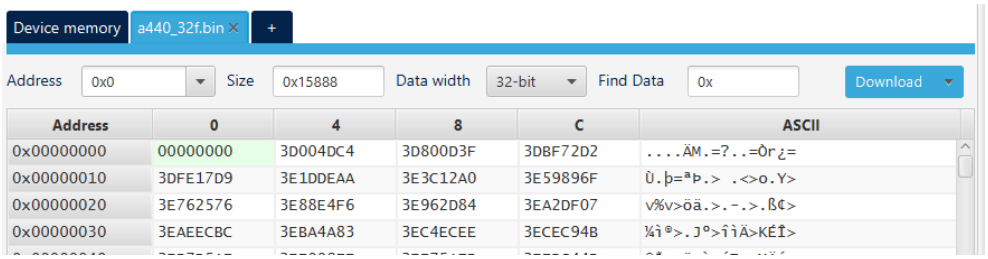
3. PREPARATION

Firstly, create a new project based on the setup for Lab 1 (you'll likely want to setup whatever is needed to be able to printf, etc.).

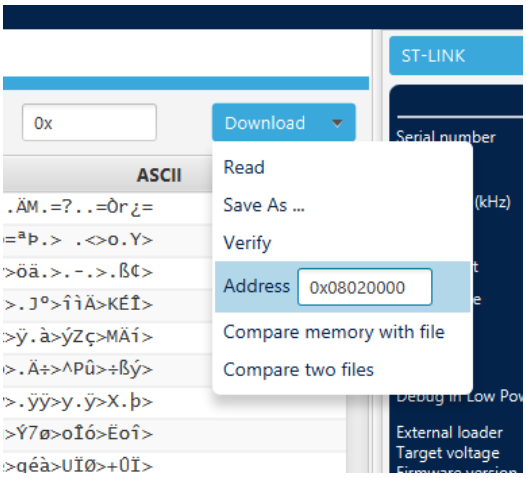
In this part of the lab, we are going to run 0.5s of a 440 Hz sine wave sampled at 44100 Hz. For our floating point experiments, we are going to use the sine wave stored as **32-bit** floats. What we need to do is get the data into our microcontroller. To do this, we will use the **STM32CubeProgrammer**. First, let's open the file **a440_32f.bin.**



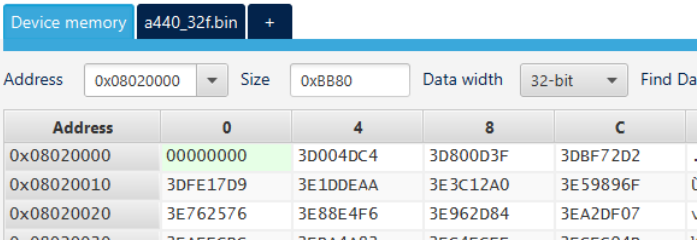
You should see the contents of that file in the window.



We are going to copy this data to a part of the microcontroller's flash memory (which should persist on reset). I've selected **0x08020000** as it's unlikely at this stage for our code to be so large as to reach this part. Select the downward arrow/triangle on "Download" and enter the address.



Now, let's connect to the board. This will only work if **you are not** currently debugging. Hit Connect on the top right. If all goes well, you board should have connected. When you hit "Download", the data should be copied to the microcontroller. You can check this by going to the Device Memory tab, and putting **0x08020000** as the address.

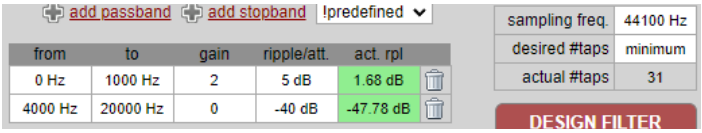


This should match up with the .bin contents!

Hit the Disconnect button on the top right (basically, we want to free up the STLink connection to the board so that the debugger in the IDE can later connect). We can now proceed to Section 7.

4. FIR FILTER – FLOATING POINT

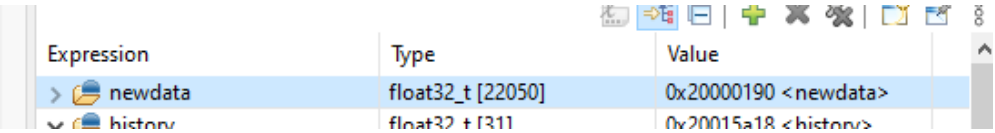
For this lab, I've used TFilter to design an FIR filter with the following characteristics.



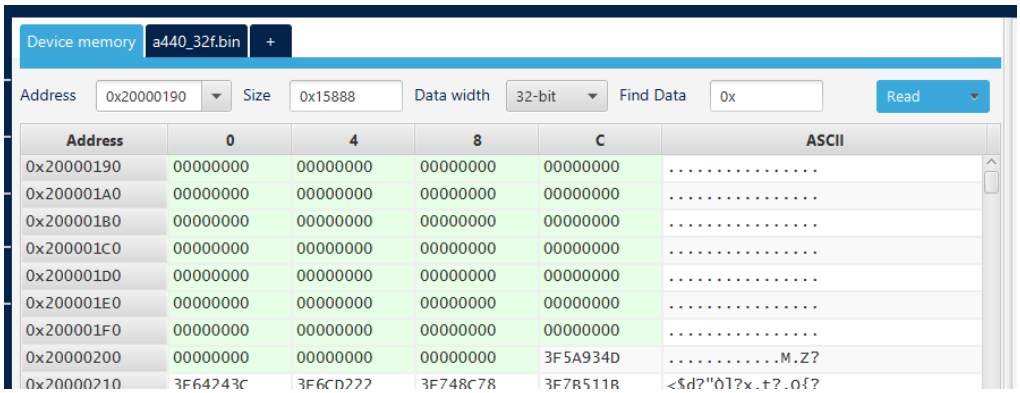
Let's run our sine wave through the filter! To do this however, we will need to write the filter function in C. There are several ways to write the C code for a filter, but in this lab, I will recommend a fairly

naïve version, where the filter coefficients and input delayed samples are stored as global variables.[1] To help you along, check out **snippets.c** for some code fragments that will help start you off. Add the various parts to your **main.c, making whatever changes as necessary.**

Once you have completed your implementation, you will need to run the debugger to find the memory location of the filtered signal. For example, my implementation has the "newdata" **at 0x20000190.**



Stop your debugging session. Let's now get the data processed data **out** of the microcontroller, again using the STM32CubeProgrammer. Connect the microcontroller. In device memory, this time set the address to where your filtered signal will end up. Set the size of memory to read accordingly. Reset the microcontroller with the black button on the board, wait a little bit, and then hit read. You should see some data! Click the down arrow on "Read" to "Save as…", extracting the transformed signal to something like filtered440_float.bin.



Q1> Implement the FIR filter (floating point) and extract the filtered signal using the STM32CubeProgrammer. Load the binary data into the Lab_Support Jupyter notebook. Plot the first 250 samples. Plot the spectrum. Screenshot these plots. How much time is taken to filter the signal? Is this filtering going to be "good enough" for our sample rate, if we were receiving new audio signals in real-time?

5. FIR FILTER – FIXED POINT

This time, you will implement the same FIR filter, but using the Q1.15 (or int16_t) data type. Follow the same process as before, **except**, I am not going to help you as much.[2] You should be able to use the model a fixed point implementation on the floating point implementation that you have just completed.

A few hints:

- You should probably comment out the floating-point related stuff from Section 7 when you embark on this part of the lab. I wouldn't delete the existing code you've written, but instead write new functions

[1] You are more than welcome to write your own version, provided you can show that you are getting the correct answers, numerically!
[2] Hint, check the lecture slides?

- You will need to use STM32CubeProgrammer to load a different input file, this time, **a440_16-bit.bin**. Note that the binary file size is different to the float version. This is still 0.5s of a 440 Hz sine wave sampled at 44100 Hz.
- For this lab, do not worry about overflow/underflow.
    - However, if you have time at the end after taking the required measurements, write code to help you count the number of times the filter output overflows/underflows

Q2> Implement the FIR filter (fixed point) and extract the filtered signal using the STM32CubeProgrammer. Load the binary data into the Lab_Support Jupyter notebook. Plot the first 250 samples. Plot the spectrum. Screenshot these plots. How much time is taken to filter the signal? Is this filtering going to be "good enough" for our sample rate in this case, if we were receiving new audio signals in real-time?

PART TWO

6. PREP

Using STM32CubeProgrammer, download `noisy.bin` to 0x08020000 to your STM32F411E DISCO, as we will use the data of noisy.bin as our input data for this lab. This file is actually audio data, sampled at 8000 Hz, and stored in the well-known `wav` format.

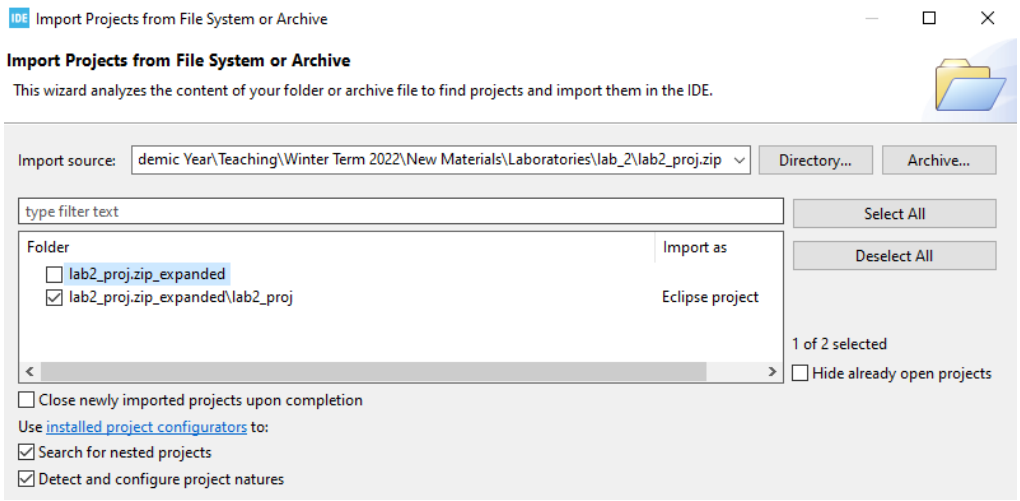
You should check out this link to get an idea of what the wav format entails, but it is, essentially, 44 bytes of header data, followed by the remaining audio data (in our case, stored as 16-bit PCM). This audio file contains both left and right channel data, so you can imagine it as something like:

L data[0]	R data[0]	L data[1]	R data[1]	L data[2]	R data[2]	….

7. IMPORT EXISTING PROJECT

For this part of the lab, you will work from a pre-existing project that we've provided. Open STM32CubeIDE and import the **lab2_proj** project into your workspace. To do this, go to File > Import, and follow the prompts. You should be able to build the project without any issues after importing. The project has a number of additional files that you can play around with outside the lab.

Note: This project is has some custom code for initializing peripherals that isn't the same as the autogenerated code that is produced by modifying the .ioc. DO NOT MODIFY THE .IOC OR AUTO-GENERATE NEW CODE.



Spend a bit of time examining the code that we've provided, focusing on **main.c**. Of particular interest are the various `#defines`, global variables, and so on.

The main fun happens line 158 onwards, in the `while(1)` loop of `int main()`.

Out of the box, you should see the following code:

```
if (sample_count < 64000) {
	newSampleL = (int16_t)raw_audio[sample_count];
	newSampleR = (int16_t)(raw_audio[sample_count] >> 16);
	sample_count++;
} else {
	sample_count = 0;
}
```

This code iterates through `raw_audio` (notice that `raw_audio` is an `int32_t*` pointing to 0x802002C). The code pulls out 2x 16-bit data at a time, separating them into `newSampleL` and `newSampleR`. In this lab, we'll focus mostly on processing `newSampleL`.

Below this is our processing code. Note that the `if` statement suppresses the filter output until we've collected enough input data.

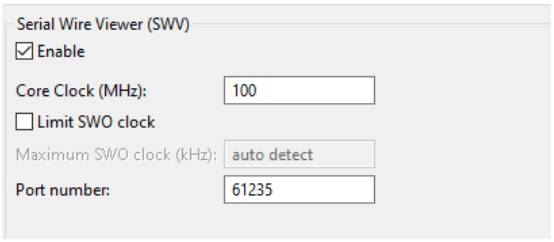
```
filteredSampleL = ProcessSample(newSampleL,history_l); // "L"
new_sample_flag = 0;
if (i < NUMBER_OF_TAPS-1) {
    filteredSampleL = 0;
    i++;
} else {
    if (bufchoice == 0) {
        filteredOutBufferA[k] = ((int32_t)filteredSampleL << 16) +
(int32_t)filteredSampleL; // copy the filtered output to both channels
    } else {
        filteredOutBufferB[k] = ((int32_t)filteredSampleL << 16) +
(int32_t)filteredSampleL;
    }

    k++;
}
```

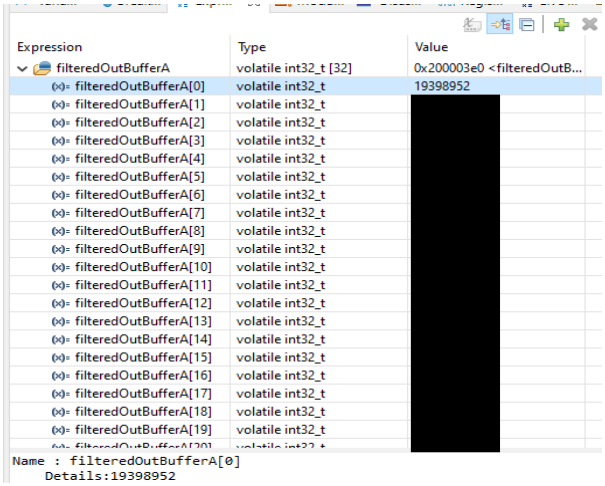
Q1> How many seconds of audio is there in the input data? Show your working.

As mentioned earlier, we're only interested in processing the L data at the moment, and we currently store the filtered output in buffers (note that we duplicate the 16-bit filtered L data to fill each entry of the filtered output buffers, where the duplicate L data is standing in for filtered R data).

Let's debug the design. Create a debug configuration as required (note that the System Core clock is configured to 100 MHz in this project). Open up the SWV Trace Log and Configure the Serial Wire Viewer settings to enable ITM Stimulus Ports 31, 30, and 0.



Q2> Run the debugger until filteredOutBufferA is full. Take a screenshot of the contents of the buffer, as below. Do you get the expected output?



Now, let's try something different. **Comment out** line 32: `#define FUNCTIONAL_TEST 1.` This will change the way our program operates, where instead of having the new data come in in the while(1) loop, the data is instead "produced" by a periodic interrupt. Check out

`void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)` near line 302.

This should trigger this call back function every 125 us. As you can see, it works essentially the same as the code in main but sets the `new_sample_flag` to 1. When the interrupt has been serviced (i.e., at the conclusion of the callback), the processor resumes its execution, probably somewhere in the `while(1)` loop of `int main()`. At some point, the `ProcessSample()` code will run because the `new_sample_flag` is now 1.

Q3> Run the debugger again until filteredOutBufferA is full. Take a screenshot of the contents of the buffer, as below. Do you get the expected output?

If we take a look at the SWV trace log, you should be able to notice that there are a lot of writes to ITM Port 30.

1	ITM Port 30	10	23800	238.000000 µs
2	ITM Port 30	10	48799	487.990000 µs
3	ITM Port 30	10	73798	737.980000 µs
4	ITM Port 30	10	98797	987.970000 µs

Take a look again at the timer callback. There following code runs if new_sample_flag is 1 when the timer callback is triggered – i.e., this is notifying us that we have missed a sample!

```
if (new_sample_flag == 1) {
	ITM_Port32(30) = 10;
}
```

8. Part Two Lab Tasks

This sets the scene for us – we want to be able to be able to filter the samples in a timely fashion! The rest of the lab is as follows:

Q4> How much time/how many cycles are required for each sample, with the original ProcessSample function? Take a screen shot of the generated assembly code and explain where you might think the main bottlenecks are.

Q5> Create a new ProcessSample2 function, making use of the appropriate MAC instruction. You might remember from class that we used in-line assembly for this. How much time is required for each sample? Do we satisfy timing requirements now? Screenshot/copy your function and explain the changes that you made.

Q6> Create a new ProcessSample3 function, making use of the appropriate SIMD instruction. You might remember from class that we might be able to use an intrinsic for this. How much time is required for each sample? Do we satisfy timing requirements now? Screenshot/copy your function and explain the changes that you made.

For each of these questions/tasks, you will need to be mindful **of checking that your code remains functionally correct**, so you might want to alternate between `#define FUNCTIONAL_TEST 1` being commented-out or defined. You can add new variables/functions as you see fit**. Be sure to add to your lab sheet explanations of what you have added or modified.**

Q7> Write a reflection of what you have observed and learned in this lab, including an explanation of how you checked that your code optimizations preserved functional correctness.

To submit:

- **Add your lab sheet and any other added/modified .c or .h files to a zip archive and upload to D2L.**